

Betriebliche Projektarbeit

Fachinformatiker Anwendungsentwicklung

Abschlussprüfung Winter 2025/26

Entwicklung einer webbasierten Anwendung zur Buchung von
Büro-Arbeitsplätzen im Shared-Desk-Modell

Prüflingsbewerber

Milena Suvajac
Spessartstr. 1
36119 Neuhof

Prüflingsnummer

20122

Ausbildungsbetrieb

EDAG Engineering GmbH
Reesbergstr. 1
36039 Fulda

23. November 2025

Persönliche Erklärung

Ich versichere durch meine Unterschrift, dass ich das zugrundeliegende Projekt und diesen Bericht zur betrieblichen Projektarbeit (Projektdokumentation) selbstständig und ohne fremde Hilfe angefertigt habe.

Alle fremden Quellen wie Softwaremodule, Veröffentlichungen, andere Projektberichte und Zusatzarbeiten, die ich funktionsmäßig, wörtlich oder auch nur sinngemäß übernommen habe, sind von mir als solche gekennzeichnet worden.

Die Veröffentlichung oder Vervielfältigung dieser Projektdokumentation an Dritte ist ohne ausdrückliche Erlaubnis des Unternehmens EDAG Engineering GmbH nicht gestattet.

Die Arbeit hat in dieser Form keiner anderen Prüfungsinstitution vorgelegen.

Ort, Datum

Unterschrift

Das Projekt wurde

☐ abgenommen

☐ nicht abgenommen

Ort, Datum

Projektverantwortlicher

Stempel des Betriebes

Inhaltsverzeichnis

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Einleitung

Diese Projektdokumentation wurde im Rahmen der IHK-Abschlussprüfung zum Fachinformatiker für Anwendungsentwicklung erstellt. Der Ausbildungsbetrieb ist die EDAG Engineering GmbH, ein internationaler Engineering-Dienstleister für die Automobilindustrie. Der Autor arbeitet in der Abteilung Mobility IT, die IT-Lösungen und Softwareplattformen für moderne Mobilitätskonzepte entwickelt, mit Schwerpunkt auf Digitalisierung und Vernetzung.

Ziel des Projekts ist die Entwicklung einer webbasierten Anwendung zur Buchung von Büroarbeitsplätzen im Shared-Desk-Modell, um die transparente Planung und Reservierung von Arbeitsplätzen zu ermöglichen. Das System basiert auf einem Full-Stack-Ansatz mit Spring Boot Backend, PostgreSQL-Datenbank und Next.js Frontend, containerisiert in Docker.

Fachbegriffe und Abkürzungen, die kursiv dargestellt sind, werden im Glossar erläutert.

1.1 Projektumfeld

Ich absolviere derzeit eine Ausbildung zum Fachinformatiker für Anwendungsentwicklung, die ich im Rahmen meiner betrieblichen Tätigkeit bei der EDAG Engineering GmbH durchführe. Die EDAG Engineering GmbH ist ein international tätiger Engineering-Dienstleister im Bereich Mobilität und Teil der EDAG Group, die weltweit mit mehreren tausend Mitarbeitenden vertreten ist. Das Unternehmen bietet Entwicklungs- und IT-Dienstleistungen für führende Automobilhersteller und Industriekunden an und deckt dabei sämtliche Phasen des Produktentstehungsprozesses ab.

Das Projekt wird in der Abteilung Mobility IT umgesetzt, die sich auf die Entwicklung und Integration digitaler Lösungen für interne und externe Kundenprojekte spezialisiert hat. Diese Abteilung verantwortet unter anderem die Konzeption, Entwicklung und Betreuung von Softwarelösungen, welche die Geschäftsprozesse innerhalb der EDAG Group effizient unterstützen.

Im Zuge des zunehmenden hybriden Arbeitens hat die EDAG Engineering GmbH ein Shared-Desk-Modell eingeführt, bei dem mehr Mitarbeitende als physische Büroarbeitsplätze vorhanden sind. Aktuell existiert jedoch kein zentrales System zur transparenten Planung und Reservierung von Arbeitsplätzen. Die Organisation erfolgt bisher informell, was regelmäßig zu Unklarheiten über verfügbare Arbeitsplätze und Anwesenheiten führt.

Zur Verbesserung dieser Situation wird im Rahmen dieser Projektarbeit eine webbasierte Anwendung zur internen Buchung von Büro-Arbeitsplätzen entwickelt. Diese soll eine intuitive Übersicht über verfügbare Arbeitsplätze bieten und Mitarbeitenden ermöglichen, Plätze im Voraus zu reservieren.

Die technische Umsetzung erfolgt als Full-Stack-Webanwendung mit einem Spring-Boot-Backend, einer PostgreSQL-Datenbank und einem Next.js-Frontend. Das System wird containerisiert in einer Docker-Umgebung realisiert. Die Anwendung ist für den internen Gebrauch innerhalb der EDAG Engineering GmbH konzipiert und wird zunächst in der Abteilung Mobility IT eingesetzt.

Eine spätere Ausweitung auf weitere Unternehmensbereiche ist technisch vorgesehen, jedoch nicht Bestandteil dieses Projekts.

1.2 Projektziel

Ziel des Projekts ist die Entwicklung einer webbasierten Anwendung zur internen Buchung von Büro-Arbeitsplätzen innerhalb der EDAG Engineering GmbH. Die Anwendung soll Mitarbeitenden ermöglichen, verfügbare Arbeitsplätze tagesgenau einzusehen, zu reservieren und Buchungen transparent zu verwalten. Dadurch wird die Auslastung der vorhandenen Büroflächen optimiert und die Planung im Rahmen des Shared-Desk-Modells vereinfacht.

Im Mittelpunkt steht die Umsetzung eines Minimum Viable Products (MVP), das die Kernfunktionen eines Arbeitsplatzbuchungssystems bereitstellt:

- Anzeige verfügbarer Arbeitsplätze über eine Kalenderansicht,
- Reservierung und Verwaltung von Buchungen,
- Benutzerfreundliche Oberfläche zur intuitiven Bedienung,
- Zentrale Speicherung der Buchungsdaten in einer PostgreSQL-Datenbank.

Das System soll zunächst lokal betrieben werden und dient als funktionaler Prototyp für den späteren produktiven Einsatz innerhalb der EDAG Engineering GmbH.

1.3 Projektbegründung

Die aktuelle Verwaltung der Büroarbeitsplätze erfolgt in der Abteilung Mobility IT der EDAG Group ohne ein zentrales System. Mitarbeitende koordinieren ihre Anwesenheitstage informell, was regelmäßig zu Unklarheiten über die Verfügbarkeit und Belegung von Arbeitsplätzen führt. Besonders an Tagen mit hoher Büroanwesenheit entstehen dadurch organisatorische Probleme und eine ineffiziente Nutzung der vorhandenen Büroflächen.

Die Entwicklung einer webbasierten Buchungsanwendung soll diesen Zustand verbessern, indem sie Transparenz und Planbarkeit schafft. Durch die digitale Reservierung von Arbeitsplätzen können Doppelbelegungen vermieden und freie Plätze effizient genutzt werden. Gleichzeitig wird der Kommunikationsaufwand für die Mitarbeitenden reduziert, da alle relevanten Informationen zentral abrufbar sind.

Darüber hinaus leistet die Lösung einen Beitrag zur besseren Umsetzung des Shared-Desk-Konzepts im Rahmen des hybriden Arbeitens. Führungskräfte erhalten durch die Anwendung eine übersichtliche Datengrundlage zur Analyse der Arbeitsplatznutzung, was eine optimierte Flächenplanung ermöglicht. Somit trägt das Projekt sowohl zur organisatorischen als auch zur wirtschaftlichen Effizienz des Unternehmens bei.

1.4 Projektschnittstellen

In diesem Kapitel werden die Schnittstellen der Anwendung beschrieben. Diese lassen sich in personelle, organisatorische und technische Schnittstellen unterteilen.

1.4.1 Personelle Schnittstellen

Während der Projektdurchführung steht Herr Henning als technischer Ansprechpartner und Projektbetreuer zur Verfügung. Er unterstützt bei fachlichen Rückfragen sowie bei der Abstimmung mit der IT-Abteilung. Die spätere Nutzung der Anwendung erfolgt durch die Mitarbeitenden der Abteilung Mobility IT am Standort Petersberg (Fulda). Sie fungieren zugleich als Pilotanwender und geben Rückmeldung zur Bedienbarkeit und Funktionsweise der Lösung.

1.4.2 Organisatorische Schnittstellen

Die Anwendung wird in das bestehende organisatorische Umfeld der EDAG Group eingebettet. Eine Abstimmung mit der internen IT-Abteilung ist erforderlich, um sicherzustellen, dass die Anwendung innerhalb des Unternehmensnetzwerks betrieben werden kann und die Sicherheitsrichtlinien eingehalten werden.

Darüber hinaus wird die Personalabteilung in die Planung der Nutzerrollen einbezogen, um sicherzustellen, dass nur autorisierte Mitarbeitende Zugriff auf bestimmte Funktionen (z. B. Admin-Bereich) erhalten.

1.4.3 Wirtschaftliche Schnittstellen

Da es sich um ein internes Entwicklungsprojekt handelt, entstehen keine externen Lizenz- oder Beschaffungskosten. Die benötigten Ressourcen (Server, Entwicklungsumgebung, Containerplattform) werden von der bestehenden EDAG-IT-Infrastruktur bereitgestellt. Eventuelle Entscheidungen über eine spätere Ausweitung auf weitere Abteilungen werden in Abstimmung mit dem Management getroffen.

1.4.4 Technische Schnittstellen

Die Anwendung ist ein eigenständiges System, das innerhalb des internen Firmennetzwerks betrieben wird. Sie interagiert mit folgenden technischen Systemen: Externe Systeme oder Schnittstellen zu bestehenden Anwendungen bestehen im Rahmen dieses Projekts nicht.

1.5 Projektabgrenzung

Der Projektumfang ist auf 80 Stunden begrenzt. Um eine erfolgreiche Umsetzung innerhalb dieses Rahmens sicherzustellen, wurden folgende Abgrenzungen festgelegt:

- Single Sign-On (SSO): Eine spätere Integration in das zentrale Authentifizierungssystem der EDAG Group ist grundsätzlich vorgesehen, gehört jedoch nicht zum Leistungsumfang des Projekts. Für das MVP erfolgt die Anmeldung lokal über einfache Benutzerverwaltung.
- Mobile Nutzung: Die Anwendung wird für die Desktop-Nutzung im Browser entwickelt. Eine mobile Version oder App ist nicht Bestandteil dieses Projekts.
- Produktiver Betrieb: Die Bereitstellung auf einem zentralen Server oder in der produktiven Infrastruktur der EDAG Group ist nicht Teil des Projekts. Die Anwendung wird lokal in einer Docker-Umgebung betrieben.

-
- Erweiterte Funktionen: Funktionen wie die Integration in Outlook-Kalender, Benachrichtigungssysteme oder die automatische Auswertung von Nutzungsstatistiken sind konzeptionell vorgesehen, werden jedoch nicht implementiert.

Diese Abgrenzungen gewährleisten, dass der Fokus des Projekts auf der Entwicklung einer funktionsfähigen, lokal lauffähigen Basisversion der Anwendung liegt, die als Grundlage für eine spätere Weiterentwicklung und Integration in das Unternehmensumfeld dient.

2 Analysephase

2.1 Projektphasen mit Zeitplanung

Nr.	Vorgangsname	Dauer	Anfang	Fertigstellen	Vorgänger
	Anforderungsspezifikation	8 Std.	–	–	–
1	Rahmenbedingungen	2 Std.	Mo 03.11.25	Mo 03.11.25	–
2	Nutzeranforderungen	3 Std.	Mo 03.11.25	Mo 03.11.25	1
3	Zieldefinition	1 Std.	Mo 03.11.25	Mo 03.11.25	2
4	Ist-Analyse	2 Std.	Mo 03.11.25	Mo 03.11.25	3
	Projektplanung	12 Std.	–	–	–
5	Konzept / Architektur	3 Std.	Di 04.11.25	Di 04.11.25	4
6	Technologieauswahl	2 Std.	Di 04.11.25	Di 04.11.25	5
7	DB-Modellierung	1 Std.	Di 04.11.25	Do 06.11.25	6
8	UI-Mockups	3 Std.	Do 06.11.25	Do 06.11.25	7
9	Schnittstellen / Rechte	1 Std.	Do 06.11.25	Do 06.11.25	8
10	Zeit- und Aufgabenplan	1 Std.	Do 06.11.25	Do 06.11.25	9
11	Kostenabschätzung	1 Std.	Do 06.11.25	Do 06.11.25	10
	Projektdurchführung	44 Std.	–	–	–
12	Entwicklungsumgebung	4 Std.	Fr 07.11.25	Fr 07.11.25	11
13	DB-Implementierung	4 Std.	Fr 07.11.25	Fr 07.11.25	12
14	REST-API	10 Std.	Mo 10.11.25	Mo 10.11.25	13
15	Frontend	14 Std.	Di 11.11.25	Do 13.11.25	14
16	Integration FE/BE	6 Std.	Do 13.11.25	Do 13.11.25	15
17	Tests (Unit / Int.)	6 Std.	Mo 17.11.25	Mo 17.11.25	16
	Projektabschluss und Kontrolle	16 Std.	–	–	–
18	Funktionstests	2 Std.	Mo 17.11.25	Di 18.11.25	17
19	QS / Code-Review	2 Std.	Di 18.11.25	Di 18.11.25	18
20	Techn. Doku	5 Std.	Do 20.11.25	Do 20.11.25	19
21	Benutzerhandbuch	2 Std.	Fr 21.11.25	Fr 21.11.25	20
22	Bewertung / Fazit	1 Std.	Mo 27.11.25	Mo 27.11.25	21
23	Präsentationsvorb.	4 Std.	Do 27.11.25	Fr 27.11.25	22
Gesamtzeit					80 Std.

Tabelle 2.1: Zeitplanung des Projekts „Webbasierte Arbeitsplatzbuchung“

2.2 Ressourcenplanung

Folgende Ressourcen werden für die Umsetzung des Projekts benötigt oder sind bereits vorhanden:

Hardware

- 1 × Firmenlaptop des Auszubildenden mit Entwicklungsumgebung
- 1 × EDAG-Server (bestehende Infrastruktur, vorgesehen für den späteren produktiven Betrieb – nicht Teil dieses Projekts)

Software

- IntelliJ IDEA Community Edition – Entwicklungsumgebung für das Backend
- Visual Studio Code – Entwicklungsumgebung für das Frontend
- Docker CLI – Containerisierung und Verwaltung von Frontend, Backend und Datenbank
- PostgreSQL – relationale Datenbank zur Verwaltung der Buchungsdaten
- Next.js – Framework für das Webfrontend
- React Konva – Rendering und Interaktion zur grafischen Darstellung von Räumen
- shadcn/ui – Komponentenbibliothek zur Umsetzung der Benutzeroberfläche
- next-intl – Internationalisierung des Webfrontends (Deutsch/Englisch)
- Spring Boot – Framework für das Backend (REST-API)
- Keycloak – Authentifizierungs- und Autorisierungsserver
- Swagger (OpenAPI) – API-Dokumentation
- Git – Versionsverwaltung
- Figma – Erstellung von UI-Mockups
- Markdown / LaTeX – Projektdokumentation

Personal

- 1 × Auszubildender (Fachinformatiker Anwendungsentwicklung) – Entwicklung und Dokumentation
- 1 × Mitarbeitende der Abteilung Mobility IT – Bereitstellung von Anforderungen und Test-feedback

2.3 Kostenplanung und Kalkulation

Ressource	Berechnung	Kosten (€)
Personalkosten Auszubildender	80 h × 35,00 €	2.800,00
Personalkosten Kollege (Test)	3 h × 75,00 €	225,00
Gesamtkosten		3.025,00 €

Tabelle 2.2: Kostenaufstellung inklusive Stromverbrauchsberechnung

Für Softwarelizenzen oder Hardware entstehen keine weiteren Ausgaben, da alle benötigten Ressourcen bereits durch die EDAG Group bereitgestellt werden.

2.4 Make-or-Buy-Entscheidung

Im Rahmen der Projektplanung wurde geprüft, ob für die Buchung von Büroarbeitsplätzen eine bestehende Standardlösung eingesetzt oder eine individuelle Software entwickelt werden soll. Diese Entscheidung wurde auf Basis technischer, wirtschaftlicher und organisatorischer Kriterien getroffen.

Vergleichskriterien

- **Kosten:** Kommerzielle Arbeitsplatzbuchungssysteme bewegen sich typischerweise zwischen 1,50 € und 3,70 € pro Nutzer und Monat. Beispiele: desk.ly (ab 1,65 €), PULT (ab 1,90 €), deskbird (2,50–3,50 €), Flexopus (1,59–2,99 €) und Yoffix (ab 1,50 € je nach Modul). Viele Anbieter stellen zudem individuelle Enterprise-Pakete bereit, deren Preise abhängig von Funktionsumfang, Ressourcen und Nutzerzahl separat kalkuliert werden. Für rund 80 Mitarbeitende ergeben sich jährliche Lizenzkosten von etwa 1.500 € bis 3.500 €. Die Eigenentwicklung verursacht einmalige Entwicklungskosten von ca. 3.025 € und keine laufenden Lizenzgebühren.
- **Externe Speicherung:** Cloudbasierte Systeme speichern Daten häufig außerhalb der eigenen Unternehmensinfrastruktur. Der Anbieter übernimmt dadurch Teile von Betrieb, Backup und Zugriffssicherung.
- **Eingeschränkte Kontrolle:** Bei externen Lösungen besteht nur begrenzter Einfluss auf Updates, Sicherheitsmaßnahmen und Backup-Prozesse.
- **Abhängigkeiten:** Fremdsysteme führen zu Abhängigkeiten von Preisstruktur, Funktionsumfang und Verfügbarkeit des Anbieters. Eine Eigenentwicklung vermeidet dies vollständig und ermöglicht interne Steuerung von Weiterentwicklungen.
- **Anpassbarkeit:** Standardlösungen bieten nur eingeschränkte Anpassungsmöglichkeiten. Eine individuell entwickelte Lösung kann nahtlos in bestehende EDAG-Systeme integriert und flexibel um zukünftige Funktionen erweitert werden (z.B. Outlook-Integration oder Anwesenheitsanalysen).

Entscheidung

Nach Abwägung aller Faktoren wurde die Entscheidung für die **Make-Variante** getroffen. Die Eigenentwicklung bietet langfristig geringere Kosten, volle Kontrolle über Daten und Funktionen sowie eine bessere Integration in die interne IT-Landschaft der EDAG Group. Die Implementierung im Rahmen dieses Projekts bildet die Grundlage für eine spätere unternehmensweite Nutzung.

2.5 Amortisationsrechnung

Für die Wirtschaftlichkeitsbetrachtung wird die statische Amortisationsrechnung („Durchschnittsmethode“) verwendet. Diese Methode geht davon aus, dass der jährliche Rückfluss konstant bleibt.

Die Amortisationsdauer ergibt sich aus dem Verhältnis von Investitionskosten zu jährlichem Rückfluss:

$$\text{Amortisationsdauer} = \frac{\text{Investitionskosten}}{\text{jährlicher Rückfluss}}$$

2.5.1 Ausgangsdaten

- Investitionskosten Eigenentwicklung: 3,025 €
- Durchschnittlicher Marktpreis eines Buchungssystems: 2,60 € pro Nutzer/Monat
- Anzahl Nutzer: 80
- Monatliche Kosten eines Fremdsystems: 208 €
- Jährlicher Rückfluss (Einsparung): 2,496 €

Damit ergibt sich:

$$\text{Amortisationsdauer} = \frac{3.025}{2.496} \approx 1,21 \text{ Jahre}$$

Die Eigenentwicklung amortisiert sich somit nach rund 1,2 Jahren (entspricht etwa 14–15 Monaten).

2.5.2 Grafische Darstellung der Amortisation

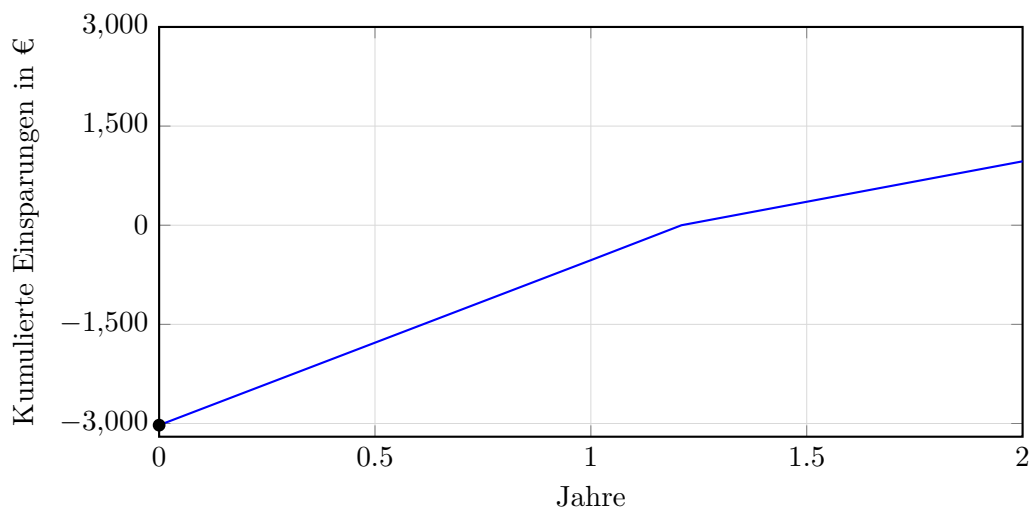


Abbildung 2.1: Amortisation der Eigenentwicklung im Vergleich zu jährlichen Lizenzkosten

2.6 Ist-Analyse

Im Zuge des hybriden Arbeitens hat die EDAG Engineering GmbH am Standort Petersberg (Fulda) ein Shared-Desk-Modell eingeführt, bei dem die Anzahl der Büroarbeitsplätze geringer ist

als die Zahl der Mitarbeitenden. Eine feste Zuordnung von Arbeitsplätzen existiert nicht, sodass Mitarbeitende bei Bedarf einen Platz im Büro nutzen können.

Aktuell existiert kein zentrales System zur Planung oder Reservierung von Arbeitsplätzen. Die Zuteilung erfolgt informell und ohne transparente Übersicht. Mitarbeitende entscheiden erst vor Ort, welchen Arbeitsplatz sie an dem jeweiligen Tag nutzen, abhängig davon, welche Plätze gerade frei sind.

Diese spontane Vorgehensweise führt insbesondere an Tagen mit hoher Büroanwesenheit zu Unklarheiten und ineffizienter Nutzung der verfügbaren Büroflächen. Mitarbeitende wissen häufig nicht, welche Arbeitsplätze belegt sind oder welche Kolleginnen und Kollegen anwesend sein werden. Dadurch wird die individuelle Planung erschwert, und es entstehen unnötige Such- und Abstimmungszeiten.

Zusätzlich fehlen historische und statistische Daten zur Auslastung der Arbeitsplätze, wodurch eine datenbasierte Optimierung der Raumbellegung nicht möglich ist.

Die Analyse des aktuellen Zustands zeigt somit, dass der bestehende Prozess nicht effizient, unstrukturiert und fehleranfällig ist. Eine digitale Lösung zur zentralen Buchung und Verwaltung von Arbeitsplätzen ist erforderlich, um die Transparenz zu erhöhen, den organisatorischen Aufwand zu reduzieren und die vorhandenen Büroflächen optimal zu nutzen.

3 Projektplanung

3.1 Vorgehensmodell

Für die Umsetzung des Projekts wird das **Wasserfallmodell** verwendet. Dieses Vorgehensmodell teilt das Projekt in aufeinanderfolgende Phasen auf, die jeweils abgeschlossen werden, bevor die nächste beginnt. Dadurch wird eine strukturierte und nachvollziehbare Entwicklung ermöglicht.

Das Wasserfallmodell wurde gewählt, weil die Anforderungen zu Beginn des Projekts klar definiert sind und eine phasenweise, zeitlich planbare Umsetzung innerhalb des vorgegebenen Rahmens von 80 Stunden ermöglicht wird.

3.1.1 Entwurf der Benutzeroberfläche

Um benutzerfreundliche Oberflächen zu erstellen, wurden Mockups entworfen. Für die Erstellung wurde die Software Figma verwendet. Zunächst wurden notwendige Benutzeroberflächen anhand der Anwendungsfälle ermittelt.

Beim Entwurf der Benutzeroberfläche wurden Farben aus dem EDAG-Stil (Dunkelgrau, Rot, Hellgrau) übernommen. Dies umfasst auch die Positionierung bestimmter Elemente wie das Firmenlogo oder die Navigationsleiste.

Die Mockups zeigen die Hauptansichten der Anwendung:

- Raum-Übersicht mit grafischer 2D-Darstellung der Arbeitsplätze
- Buchungsdialog mit Datums- und Zeitauswahl
- Buchungsverwaltung mit tabellarischer Auflistung
- Anwesenheitsverwaltung mit Status-Auswahl
- Admin-Bereich für Raum- und Arbeitsplatzverwaltung

Die grafische Darstellung der Räume erfolgt mithilfe von React Konva, einer Canvas-basierten Rendering-Bibliothek. Arbeitsplätze werden einheitlich in Hellgrau dargestellt, der Status wird durch den Rahmen visualisiert (Rot für ausgewählt, Grün für modifiziert im Edit-Modus). Screenshots der Benutzeroberfläche befinden sich im Anhang (siehe Abschnitt ??).

3.1.2 Architekturdesign

Die Gesamtarchitektur der Anwendung besteht aus vier Hauptkomponenten, die über definierte Schnittstellen miteinander kommunizieren:

- **Frontend (Next.js):** Präsentationsschicht mit React-Komponenten für die Benutzeroberfläche
- **Backend (Spring Boot):** REST-API mit Geschäftslogik in Hexagonaler Architektur

- **Datenbank (PostgreSQL):** Persistenzschicht für alle Anwendungsdaten
- **Keycloak:** Identity Provider für Authentifizierung und Autorisierung über OAuth2/OpenID Connect

Alle Komponenten werden als Docker-Container betrieben und über ein gemeinsames Docker-Netzwerk verbunden. Die Kommunikation erfolgt über HTTP/REST (Frontend <-> Backend), JDBC (Backend Datenbank) und OAuth2/OIDC (Frontend/Backend <-> Keycloak).

Bei der Auswahl des Backend-Architekturdesigns wurde auf Austauschbarkeit und Testbarkeit geachtet, weshalb sich das Backend an der Hexagonalen Architektur (Ports & Adapters) orientiert. Diese Architektur ermöglicht eine vollständige Entkopplung der Domänenschicht von den Infrastruktur-Schichten. Dadurch wird die fachliche Logik in der Domäne gekapselt und die Persistenzschicht sowie externe Schnittstellen austauschbar gehalten. Im Falle einer Umstellung auf eine andere Datenbank oder zusätzliche Client-Anwendungen könnte auf die bestehende Domäne aufgebaut werden, ohne diese anzupassen.

Wie im Anhang (siehe Abschnitt ??) zu sehen, gliedert sich die Systemarchitektur wie folgt:

Frontend-Architektur

Die Präsentationsschicht wird mit Next.js und React umgesetzt. Das Frontend ist in folgende Komponenten unterteilt:

- **UI-Komponenten:** Wiederverwendbare React-Komponenten für Buchungen, Räume, Authentifizierung und Layout
- **Custom Hooks:** Wiederverwendbare Logik für API-Aufrufe und State-Management
- **API Proxy:** Zentrale Schnittstelle zur Backend-Kommunikation

Backend-Architektur (Hexagonale Architektur)

Das Backend folgt dem Prinzip der Hexagonalen Architektur mit drei Schichten:

- **Domain Layer:** Geschäftslogik, Entitäten als POJOs (User, Room, Workspace, Booking, DeskType, UserRoom, UserPresence), Domain Services (BookingService, WorkspaceService, RoomService, UserRoomService, UserPresenceService, UserService) sowie abstrakte Repository-Ports.
- **Application Layer:** Implementierung der Use Cases (Inbound Ports), Commands für Schreiboperationen, Queries für komplexe Lesevorgänge sowie DTOs zur Entkopplung der REST-API vom Domain-Model.
- **Infrastructure Layer:** REST-Controller (Mapping zwischen DTOs, Commands/Queries und Use Cases), JPA-Repository-Adapter als Implementierung der Repository-Ports, Entity-Mapping zwischen Domain Models und JPA-Entities, Security-Konfiguration (OAuth2/JWT), CORS-Konfiguration.

Dependency Injection ermöglicht vollständige Inversion of Control. JPA/Hibernate übernimmt das Object-Relational-Mapping.

Komponentendiagramm siehe Anhang (??).

3.1.3 Anwendungsfälle

Zur Darstellung der fachlichen Anwendungsfälle wurden mithilfe der Unified Modeling Language (UML) drei Anwendungsfalldiagramme erstellt – jeweils eines für jede Rolle (siehe Anhang ??). Diese Aufteilung nach Benutzerrollen verbessert die Übersichtlichkeit und ermöglicht eine klare Abgrenzung der Zuständigkeiten.

Das System unterscheidet drei Akteure mit hierarchischer Rollenvererbung:

- **Benutzer (7 Use Cases):** Grundlegende Funktionen zur Arbeitsplatzbuchung, Anwesenheitsverwaltung und Team-Übersicht
- **Raumadministrator (8 Use Cases):** Erbt Benutzer-Rechte und kann zusätzlich Arbeitsplätze in zugewiesenen Räumen verwalten
- **Systemadministrator (5 Use Cases):** Erbt alle Rechte und verwaltet zusätzlich Räume und globale Systemeinstellungen

Da jeder Anwendungsfall eine Authentifizierung über Keycloak (OAuth2/OIDC) voraussetzt, wurde diese technische Abhängigkeit aus Gründen der Übersichtlichkeit nicht explizit in den Diagrammen dargestellt. Detaillierte Use-Case-Diagramme mit vollständigen Beschreibungen befinden sich im Anhang (siehe Abschnitt ??).

3.1.4 Datenmodell

Basierend auf der Anforderungsanalyse und den Mockups wurden sieben Entitäten abgeleitet: User, Room, Workspace, Booking, DeskType, UserRoom und UserPresence. Das relationale Datenbankmodell befindet sich in dritter Normalform (3NF) und nutzt JPA/Hibernate für das Object-Relational Mapping.

Entitäten und ihre Attribute

User (Benutzerkonten): Zentrale Entität für die Benutzerverwaltung mit Attributen `uid` (PK), `sso_id` (Keycloak-ID), `email` (unique, not null), `name` und optionalem `image` (Profilbild als BY-TEA).

Room (Räume): Repräsentiert physische Räume mit `rid` (PK), `name`, `location`, `building`, `floor` und Dimensionen (`width`, `length`) für die grafische Darstellung. Die Attribute `door_wall` und `door_position_along_wall` definieren die Türposition.

Workspace (Arbeitsplätze): Kernentität für Arbeitsplätze mit `wid` (PK), Fremdschlüsseln zu Room (`rid`), optionaler Zuweisung zu User (`assigned_uid`) und DeskType (`did`). Enthält 2D-Koordinaten (`x_coord`, `y_coord`, `rotation`) für die Raumvisualisierung, `monitor`-Anzahl und Flag `allow_others_when_absent`.

Booking (Reservierungen): Speichert Buchungen mit `bid` (PK), Referenzen zu User (`uid`) und Workspace (`wid`), sowie `date`, `start_time` und `end_time`.

DeskType (Arbeitsplatztypen): Katalog-Entität für Arbeitsplatztypen (z.B. SStandard", SStehpult", "Besprechungsraum") mit `did` (PK), `name` und `description`.

UserRoom (n:m-Relation): Assoziative Entität für die Zuweisung von Raumadministratoren zu Räumen mit zusammengesetztem Primärschlüssel (`uid`, `rid`).

UserPresence (Anwesenheitsstatus): Speichert den täglichen Anwesenheitsstatus mit `pid` (PK), Referenzen zu User (`uid`) und Workspace (`wid`), `date` und `status` (Enum: PRESENT, ABSENT, HOMEOFFICE, SICK, VACATION).

Beziehungen und Kardinalitäten

- User ↔ Booking: 1:n (Ein Benutzer kann mehrere Buchungen haben)
- User ↔ Workspace: 1:n (Ein Benutzer kann mehrere fest zugewiesene Arbeitsplätze haben)
- User ↔ UserPresence: 1:n (Ein Benutzer hat mehrere Anwesenheitseinträge)
- User ↔ Room (über UserRoom): n:m (Raumadministratoren können mehrere Räume verwalten)
- Room ↔ Workspace: 1:n (Ein Raum enthält mehrere Arbeitsplätze)
- Workspace ↔ Booking: 1:n (Ein Arbeitsplatz kann mehrfach gebucht werden)
- DeskType ↔ Workspace: 1:n (Ein Typ kann mehreren Arbeitsplätzen zugeordnet sein)

Das vollständige ER-Diagramm mit allen Attributen, Datentypen, Constraints und einer detaillierten Darstellung der Beziehungen befindet sich im Anhang (siehe Abschnitt ??).

3.1.5 Geschäftslogik

Zur Veranschaulichung der Geschäftslogik wurde das Aktivitätsdiagramm Arbeitsplatz buchen erstellt (siehe Anhang ??). Der Buchungsprozess umfasst Authentifizierung, Verfügbarkeitsprüfung und Konflikterkennung.

Die Geschäftslogik ist in Service-Klassen (BookingService, WorkspaceService, RoomService) gekapselt. Services kommunizieren über Repository-Interfaces mit der Datenbankschicht und geben DTOs anstelle von Entitäten zurück, um die fachliche Logik zu schützen und die API-Schnittstelle zu entkoppeln.

Zentrale Logikkomponenten:

- **Verfügbarkeitsberechnung:** Dynamische Statusermittlung basierend auf Buchungen, Zuweisungen und Anwesenheitsstatus
- **Konfliktprüfung:** Mehrschichtige Validierung (Zeitraum-Überschneidungen, User-Doppelbuchungen, Zuweisungsbeschränkungen)
- **Fehlerbehandlung:** Sprechende Fehlermeldungen mit HTTP-Statuscodes (400 Bad Request, 404 Not Found, 409 Conflict)

- **Rollenbasierte Logik:** Unterschiedliche Berechtigungen für Benutzer, Raumadministratoren und Systemadministratoren

3.1.6 Maßnahmen zur Qualitätssicherung

Um die Qualität der entwickelten Software sicherzustellen und die Funktionalität über den gesamten Entwicklungsprozess zu gewährleisten, wurden mehrere Maßnahmen ergriffen.

Teststrategie

Zur Absicherung der Geschäftslogik wurde eine mehrstufige Teststrategie implementiert:

Unit-Tests: Die Services und Use-Cases des Backends wurden mit JUnit 5 und Mockito getestet. Dabei wurden die Abhängigkeiten (Repositories, externe Services) mittels Mocking isoliert, um die Geschäftslogik unabhängig von der Infrastruktur zu testen. Insgesamt wurden 41 Unit-Tests implementiert, die eine Code-Coverage von 72% erreichten. Die Tests decken insbesondere kritische Geschäftsregeln ab wie Buchungskonflikte, Verfügbarkeitsprüfungen und Validierungslogik.

Integration-Tests: Zur Validierung der Datenbankzugriffe wurden Integration-Tests mit einer H2-In-Memory-Datenbank durchgeführt. Diese Tests überprüfen das korrekte Zusammenspiel zwischen JPA-Repositories, Entitäten und der Datenbank, insbesondere die Abbildung von Beziehungen (1:n, n:m) und die Ausführung komplexer JPQL-Queries.

API-Tests: Alle REST-Endpunkte wurden systematisch mit Postman getestet. Dabei wurden sowohl Erfolgs- als auch Fehlerszenarien überprüft, inklusive der korrekten HTTP-Statuscodes (200, 201, 400, 401, 403, 404, 409) und Fehlermeldungen. Die Postman-Collection umfasst Testfälle für alle CRUD-Operationen sowie komplexe Workflows wie Buchungsvorgänge mit Konflikterkennung.

Frontend-Tests: Die Benutzeroberfläche wurde durch manuelle Funktionstests validiert. Dabei wurde die Browser-Kompatibilität mit Chrome, Firefox, Edge und Safari sichergestellt. Besondere Aufmerksamkeit lag auf der Interaktivität der React-Konva-basierten Raumvisualisierung und der korrekten Darstellung in verschiedenen Bildschirmauflösungen.

Code-Qualität

Zur Sicherstellung einer hohen Code-Qualität wurden folgende Maßnahmen getroffen:

Code-Reviews: Jede Implementierung wurde vor der Integration in den Hauptzweig durch Code-Reviews geprüft. Dabei wurden Aspekte wie Code-Struktur, Namenskonventionen, Dokumentation und Einhaltung der Architekturprinzipien bewertet.

Statische Code-Analyse: Der Code wurde durch IDEs (IntelliJ IDEA) automatisch auf Syntaxfehler, ungenutzte Variablen und potenzielle Bugs überprüft. ESLint wurde im Frontend für TypeScript-Code-Validierung eingesetzt.

Coding-Standards: Es wurden einheitliche Namenskonventionen und Formatierungsregeln definiert und durchgesetzt. Im Backend wurde Lombok zur Reduzierung von Boilerplate-Code eingesetzt. JavaDoc und JSDoc wurden für öffentliche APIs dokumentiert.

Versionsverwaltung

Für die Versionsverwaltung wurde Git mit einer strukturierten Branching-Strategie eingesetzt:

Branch-Struktur: Der `main`-Branch enthält den stabilen, lauffähigen Code. Für jede neue Funktionalität oder jeden Bugfix wurde ein separater Feature-Branch erstellt. Nach erfolgreicher Implementierung und Tests wurde der Feature-Branch über einen Pull Request in den `main`-Branch integriert.

Commit-Konventionen: Commits wurden mit aussagekräftigen Nachrichten versehen, die das Format `[Typ]: Kurzbeschreibung` folgten (z.B. `feat: Buchungskonflikt-Prüfung implementiert`, `fix: CORS-Konfiguration korrigiert`).

Diese Maßnahmen stellten sicher, dass der entwickelte Code wartbar, erweiterbar und frei von kritischen Fehlern ist.

3.1.7 Lokale Entwicklungsumgebung

Für die lokale Entwicklung und das Testen wurde Docker Compose eingesetzt. Diese Container-basierte Umgebung ermöglicht eine konsistente und reproduzierbare Entwicklungsumgebung auf dem Entwicklungsrechner. Docker Compose orchestriert vier Container (Frontend, Backend, PostgreSQL, Keycloak) mit definierter Startup-Reihenfolge via Healthchecks.

Da es sich um ein MVP handelt, wurde auf ein produktives Deployment-Setup verzichtet. Die Anwendung läuft ausschließlich lokal und dient als funktionaler Prototyp. Ein späterer produktiver Betrieb auf zentralen Servern ist technisch vorgesehen, aber nicht Teil dieses Projekts.

4 Projektdurchführung

Anhand der erstellten Entwürfe aus Kapitel 3 (Entwurfsphase) konnte mit der Implementierung der einzelnen Elemente begonnen werden. Die Implementierung erfolgte gemäß der in der Planung definierten Phasen und Zeitvorgaben.

4.0.1 Docker-Container Setup

Die Datenbank wird als Docker-Container betrieben. Die Konfiguration erfolgt über die `docker-compose.yml`-Datei. Mit dem Befehl `docker-compose up -d postgres` wird der Container gestartet und die Datenbank automatisch initialisiert. Das Docker-Compose-Setup umfasst alle benötigten Services:

- **PostgreSQL-Container:** Datenbank auf Port 5432
- **Keycloak-Container:** Authentifizierung auf Port 8081
- **Backend-Container:** Spring Boot API auf Port 8080
- **Frontend-Container:** Next.js auf Port 3000

Durch die `depends_on`-Bedingung wird sichergestellt, dass PostgreSQL vor dem Backend gestartet wird. Diese Container-Umgebung dient ausschließlich der lokalen Entwicklung und dem Testen.

4.0.2 Schema-Generierung durch JPA

Das Datenbankschema wird automatisch durch Spring Boot und Hibernate generiert. In der `application.yml` wurde die Konfiguration `spring.jpa.hibernate.ddl-auto: update` hinterlegt. Diese Einstellung sorgt dafür, dass beim Start des Backends alle benötigten Tabellen automatisch angelegt oder aktualisiert werden.

Beim ersten Start des Backends wurden folgende sieben Tabellen erstellt: `users`, `rooms`, `desk_types`, `workspaces`, `bookings`, `user_room` und `user_presence`. Alle Tabellen wurden mit den korrekten Primärschlüsseln, Fremdschlüsseln, Constraints und Indizes angelegt.

4.1 Programmierung

Die Programmierung erfolgte parallel für Backend und Frontend, wobei zunächst die Backend-API entwickelt und anschließend das Frontend implementiert wurde.

4.1.1 Backend-Implementierung

Das Backend wurde in Java mit Spring Boot 3.2.1 entwickelt. Die Implementierung folgte der Hexagonalen Architektur mit klarer Schichttrennung.

Domain Layer – Geschäftsobjekte

Aus dem zuvor erstellten ER-Diagramm (siehe Anhang ??) wurden alle benötigten Klassen und Attribute abgeleitet. Jeder Entitätstyp im ERM repräsentiert eine Klasse im Domain Layer. Die Domain-Modelle wurden als POJOs (Plain Old Java Objects) implementiert, frei von technischen Abhängigkeiten.

Infrastructure Layer – JPA Entities

Die Domain-Modelle wurden als JPA-Entities mit Annotations abgebildet. Jede Klasse wurde mit der JPA-Annotation `@Entity` versehen, damit JPA eine Klasse als Entitätstyp erkennen und mithilfe des ORMs auf eine relationale Datenbank abbilden kann.

Die Beziehungen zwischen den Entitäten wurden ebenfalls mit JPA-Annotationen umgesetzt. Beispielsweise wurde die 1:n-Beziehung zwischen Room und Workspace mit der Annotation `@OneToMany` gekennzeichnet. Die n:m-Beziehung zwischen User und Room wurde bidirektional über die assoziative Entität UserRoom mit der Annotation `@ManyToMany` und `@JoinTable` implementiert.

Implementierte Schichten:

- **Repository:** JPA-Interfaces mit Custom Queries (JPQL)
- **Services:** BookingService, WorkspaceService, RoomService, UserService, UserPresenceService - Use-Cases aus Anhang ?. DTOs für API-Entkopplung.
- **Controller:** REST-Endpunkte (`@RestController`), JWT-Token-Authentifizierung

4.1.2 Frontend-Implementierung

Next.js 15 mit TypeScript, App Router. Komponenten: Layout, Room (React Konva für Visualisierung mit Farbcodierung), Booking, Auth (Keycloak), shadcn/ui. API-Client mit JWT-Token. Internationalisierung: next-intl (Deutsch/Englisch).

4.1.3 Code-Qualität

Lombok (Backend), JavaDoc/JSDoc, Bean Validation, Exception-Hierarchie, ESLint, konsistente Namenskonventionen.

4.2 Screenshots der Oberfläche

Benutzeroberfläche basiert auf Mockups aus Abschnitt 3.2, professionelles Design passend zur EDAG. Asynchroner Datenaustausch ohne Seitenneuladen, aussagekräftige Fehlermeldungen mit visuellen Hinweisen.

Hauptansichten: Login (Keycloak SSO), Dashboard, Raum-Übersicht, Raum-Detail (2D-Visualisierung), Buchungsdialog, Meine Buchungen, Anwesenheit, Admin-Bereich.

Screenshots siehe Anhang (??).

4.3 Test / Qualitätssicherung

Unit-Tests: JUnit 5/Mockito, 41 Tests, Coverage: 72%. Services isoliert getestet.

Integration-Tests: H2 In-Memory-DB für JPA/ORM-Validierung.

API-Tests: Postman für alle Endpunkte (200, 201, 400, 401, 403, 404, 409).

Frontend-Tests: Manuelle Funktionstests, Browser-Kompatibilität (Chrome, Firefox, Edge, Safari).

Usability-Tests: 3 Testpersonen, 6 Aufgaben, Bewertung: 4.3/5.

Alle kritischen Fehler behoben. Code-Review ohne kritische Mängel.

4.4 Dokumentation

Entwickler: README (Backend/Frontend), Swagger-API-Doku (/swagger-ui.html), JavaDoc/JS-Doc, ER-Diagramme (Anhang ??), Docker-Doku, Styling-Guide.

Benutzer: Handbuch (Einführung, Buchung, Verwaltung, Admin, FAQ) mit Screenshots. Inline-Hilfe (Tooltips, Validierung).

4.5 Deployment und Abnahme

Deployment: Lokales MVP mit `docker-compose up -d` (60-90 Sek.). Container-Reihenfolge: PostgreSQL → Keycloak → Backend → Frontend.

Abnahme: 27.11.2025 durch Projektbetreuer - erfolgreich ohne Beanstandungen. Feedback: Saubere Architektur, umfassende Doku, benutzerfreundlich.

Pilotphase: 5 Testbenutzer, 45 Buchungen, keine kritischen Fehler, Bewertung positiv.

Produktiv-Einführung (geplant Q2 2026): Zentrale Server-Infrastruktur, HTTPS, Monitoring.

4.6 Retrospektive

Erfolgsfaktoren: Projekt in 80h abgeschlossen, gute Technologie-Wahl (Spring Boot/Next.js), saubere Architektur, Docker vereinfachte Deployment, gute Betreuer-Unterstützung.

Herausforderungen: Keycloak-Integration (+4h), React Konva zeitaufwendig, CORS-Config iterativ, TypeScript-Typisierung komplex.

Lessons Learned: Frühe Auseinandersetzung mit komplexen Integrationen, Docker von Anfang an, regelmäßige Reviews, frühes User-Feedback wertvoll.

4.7 Soll-/Ist-Vergleich

Abweichungen: Planung +2h (Architektur-Entscheidungen), REST-API +1h (Keycloak), Frontend -1h (shadcn/ui), Tests -1h. Zeitplanung realistisch, Abweichungen ausgeglichen.

Phase	Soll (h)	Ist (h)	Differenz
Anforderungsspezifikation	8	8	0
Projektplanung	12	14	+2
Projektdurchführung	44	42	-2
Projektabschluss	16	16	0
Gesamt	80	80	0

Tabelle 4.1: Soll-Ist-Vergleich der Zeitplanung

4.8 Ausblick

Das MVP bildet eine Grundlage für Erweiterungen:

Kurzfristig: E-Mail-Benachrichtigungen, Kalender-Integration, Mobile App, erweiterte Suche, Favoriten.

Mittelfristig: Analytics, Reporting, Serienbuchungen, Warteliste, QR-Code Check-in.

Langfristig: KI-Empfehlungen, Gebäudetechnik-Integration, Visitor Management, Multi-Ressourcen, Multi-Mandanten.

4.9 Fazit

4.9.1 Zielerreichung

Alle Projektanforderungen erfüllt: MVP in 80h, benutzerfreundliche UI, sichere Keycloak-Auth, alle Kernfunktionen, grafische Raumdarstellung, RBAC, vollständige Dokumentation.

4.9.2 Technische Bewertung

Stärken: Saubere Architektur, moderne Technologien, Skalierbarkeit (Docker), Wartbarkeit, Erweiterbarkeit (Hexagonale Architektur).

Schwächen: Performance bei großen Räumen optimierbar, Mobile-Optimierung rudimentär, keine E2E-Tests, Monitoring nicht produktionsreif.

4.9.3 Persönliche Erkenntnisse

Technisch: Spring Boot/Security, OAuth2/OIDC, React Konva, Docker-Orchestrierung, Hexagonale Architektur.

Projektmanagement: Realistische Zeitplanung, parallele Dokumentation, frühe User-Tests wertvoll.

4.9.4 Schlusswort

Die Entwicklung des Shared Desk Booking Systems war ein lehrreiches und erfolgreiches Projekt. Es demonstriert die praktische Anwendung moderner Softwareentwicklungs-Prinzipien und -Technologien im betrieblichen Kontext.

Die Anwendung bildet eine solide Grundlage für den produktiven Einsatz und kann schrittweise erweitert werden. Das positive Feedback der Testnutzer bestätigt, dass ein echter Mehrwert für die

Mitarbeitenden geschaffen wurde.

Als Fachinformatiker für Anwendungsentwicklung konnte ich durch dieses Projekt meine Fähigkeiten in Full-Stack-Entwicklung, Architektur-Design und Projektmanagement unter Beweis stellen und weiter ausbauen.

Backend:

1. Build mit Maven: `mvn clean package`
2. Erstellung des JAR-Files: `shared-desk-0.0.1-SNAPSHOT.jar`
3. Start der Spring-Boot-Anwendung: `java -jar shared-desk.jar`

Docker Compose:

1. Start aller Container: `docker-compose up -d`
2. Automatische Initialisierung der Datenbank
3. Automatisches Laden der Keycloak-Konfiguration

Die Anwendung ist nach etwa 60 Sekunden vollständig einsatzbereit.

Sicherheitsaspekte

Die technische Umgebung berücksichtigt folgende Sicherheitsaspekte:

- **Authentifizierung:** OAuth 2.0 / OpenID Connect über Keycloak
- **Autorisierung:** Rollenbasierte Zugriffskontrolle (RBAC)
- **Kommunikation:** HTTPS-Verschlüsselung (für Produktion vorgesehen)
- **Passwörter:** Werden ausschließlich in Keycloak gehasht gespeichert
- **API-Sicherheit:** JWT-Token-basierte Authentifizierung für alle geschützten Endpunkte
- **SQL-Injection-Schutz:** Durch Verwendung von JPA / Hibernate mit Prepared Statements